

Introduction to DOM (Document Object Model):

- **DOM (Document Object Model)** is a **programming interface** for HTML and XML documents.
- It represents the webpage as a **tree structure** where every part of the page is a **node/object** (HTML elements, attributes, text, etc.).
- Using DOM, JavaScript can **access, modify, add, or delete** HTML elements dynamically.

It represents the page so programs (JavaScript) can **manipulate**:

- Structure (HTML tags)
- Style (CSS)
- Content (Text, attributes)
- Events (Click, input, keypress, etc.)

How DOM Works:

- Browser converts HTML → DOM Tree
- Each HTML element becomes a **node/object**
- JavaScript can access or modify any node

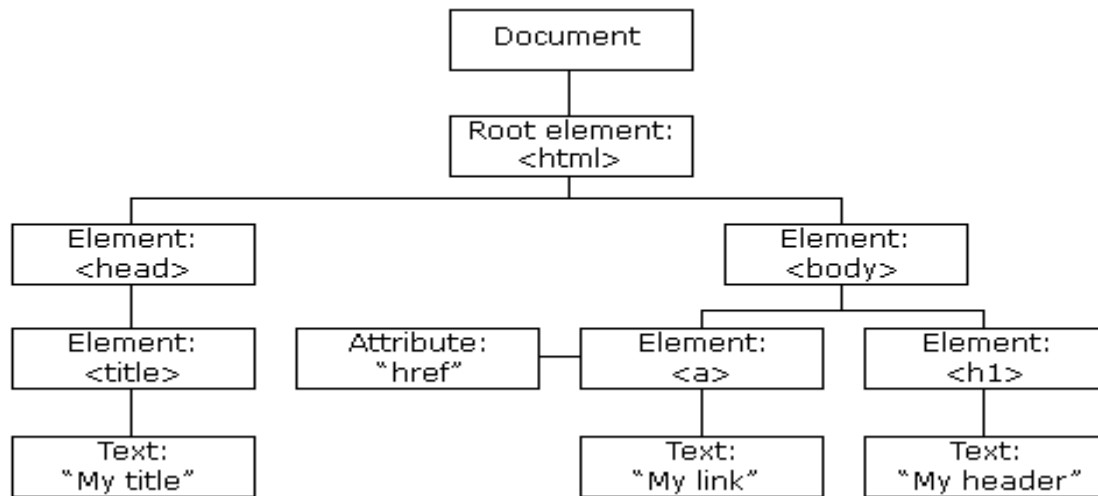
Types of Nodes:

1. **Document Node** — the root (document)
2. **Element Node** — tags (div, p)
3. **Text Node** — actual text inside elements

4. **Attribute Node** — attributes (id="", class="")

5. **Comment Node** — <!-- Comment -->

DOM Tree Structure:



Why DOM is Important:

DOM allows JavaScript to:

- Change HTML content
- Change style (CSS)
- Add/Remove HTML elements
- Handle events
- Animate elements
- Build dynamic interactive websites

DOM Properties & Methods :

1. Selecting Elements

Method	Description	Example
<code>getElementById()</code>	Selects element by ID	<code>document.getElementById("msg")</code>
<code>getElementsByClassName()</code>	Selects elements by class (HTMLCollection)	<code>document.getElementsByClassName("item")</code>
<code>getElementsByTagName()</code>	Selects elements by tag	<code>document.getElementsByTagName("p")</code>
<code>querySelector()</code>	Selects first matching element	<code>document.querySelector(".box")</code>
<code>querySelectorAll()</code>	Selects all matching elements (NodeList)	<code>document.querySelectorAll("li")</code>

2. Creating Elements

Method	Description	Example
<code>createElement()</code>	Creates a new HTML element	<code>document.createElement("div")</code>
<code>createTextNode()</code>	Creates a text node	<code>document.createTextNode("Hello")</code>

3. Adding Elements to DOM

Method	Description	Example
<code>appendChild()</code>	Adds element at the end of parent	<code>parent.appendChild(child)</code>
<code>append()</code>	Adds text or multiple nodes inside element	<code>box.append(span, "World")</code>
<code>prepend()</code>	Inserts content at beginning	<code>\list.prepend(li)</code>
<code>insertBefore()</code>	Inserts element before another	<code>parent.insertBefore(newEl, refEl)</code>

4. Removing & Replacing Elements

Method	Description	Example
<code>remove()</code>	Removes the element itself	<code>element.remove()</code>
<code>removeChild()</code>	Removes a child from parent	<code>parent.removeChild(child)</code>
<code>replaceChild()</code>	Replaces an element with another	<code>parent.replaceChild(new, old)</code>

5. Attribute Methods

Method	Description	Example
<code>getAttribute()</code>	Gets an attribute value	<code>img.getAttribute("src")</code>
<code>setAttribute()</code>	Sets/updates an attribute	<code>p.setAttribute("class", "active")</code>
<code>removeAttribute()</code>	Removes an attribute	<code>btn.removeAttribute("disabled")</code>
<code>hasAttribute()</code>	Checks if attribute exists	<code>p.hasAttribute("class")</code>

Events :

A **DOM event** is an action or occurrence that happens in the browser, such as a click, key press, mouse movement, or page load, which JavaScript can detect and respond to.

Why Events Are Important?

Events allow JavaScript to:

- Make web pages interactive
- Respond to user actions
- Validate forms
- Create dynamic UI updates
- Control animations and media

Types of Events:

- **Mouse Events** – click, dblclick, mouseover, mouseout
- **Keyboard Events** – keydown, keyup, keypress
- **Form Events** – submit, focus, change, input
- **Window Events** – load, scroll, resize

Add Event Listener

Syntax :

```
element.addEventListener("event", function);
```

Example :

```
<button id="btn">Click Me</button>
<script>
  document.getElementById("btn").addEventListener("click", () => {
    alert("Button Clicked!");
  });
</script>
```

Inline Events

```
<button onclick="show()">Click</button>
<script>
  function show() {
    console.log("Clicked!");
  }
</script>
```

Event Mechanics

1. Event Object

When an event occurs, JavaScript passes an **event object** to handler.

Example :

```
btn.addEventListener("click", function(e) {  
    console.log(e.target);  
    console.log(e.type);  
    console.log(e.clientX);  
});
```

2. Event Bubbling (Default)

Event moves from **inner** → **outer** elements.

Div → Body → Document → Window

Example :

```
<div id="outer">  
    <button id="inner">Click Me</button>  
</div>  
<script>  
    document.getElementById("outer").addEventListener("click", () => {  
        console.log("Outer Click");  
    });  
    document.getElementById("inner").addEventListener("click", () => {  
        console.log("Inner Click");  
    });  
</script>
```

3. Event Capturing (Trickling Phase)

Flow is **outer** → **inner**

Syntax :

```
addEventListener("click", handler, true);
```

4. Event Delegation

Attach event to **parent instead of all children**

Example :

```
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
<script>
document.getElementById("list").addEventListener("click", function(e){
  if(e.target.tagName === "LI"){
    console.log("Clicked:", e.target.innerText);
  }
});
</script>
```

5. stopPropagation()

Prevents event from moving to parent elements.

Example :

```
child.addEventListener("click", (e) => {
  e.stopPropagation();
  console.log("Child only");
});
```

6. preventDefault()

Prevents default browser behavior (form submit, link open, right-click menu, etc.)

Example :

```
document.querySelector("a").addEventListener("click", (e) => {
  e.preventDefault();
  alert("Link disabled!");
});
```

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM Todo App</title>
  <style>
    #outer {
      border: 2px solid blue;
      padding: 20px;
      width: 350px;
    }
    li {
      margin: 8px 0;
      padding: 6px;
      background: #f2f2f2;
      display: flex;
      justify-content: space-between;
    }
  </style>
</head>
<body>
<div id="outer">
  <h2>Todo App</h2>
  <form id="todoForm">
    <input type="text" id="todoInput" placeholder="Enter Task..." />
    <button type="submit" id="addBtn">Add</button>
  </form>
  <ul id="todoList"></ul>
</div>
<script>
const form = document.getElementById("todoForm");
const input = document.querySelector("#todoInput");
const list = document.getElementById("todoList");
const outerBox = document.getElementById("outer");

form.addEventListener("submit", function (e) {
  e.preventDefault();

  if (input.value.trim() === "") return;

  const li = document.createElement("li");    const textNode =
```

```

document.createTextNode(input.value);    const delBtn =
document.createElement("button");
    delBtn.setAttribute("class", "delete-btn");
    delBtn.textContent = "Delete";
    li.appendChild(textNode);
    li.appendChild(delBtn);
    list.append(li);
    input.value = "";
});

list.addEventListener("click", function (e) {
    console.log("Event Target:", e.target);
    console.log("Event Type:", e.type);
    console.log("Mouse Position:", e.clientX, e.clientY);
    if (e.target.classList.contains("delete-btn")) {
        e.target.parentElement.remove();
    }
});

outerBox.addEventListener("click", function () {
    console.log("Outer DIV clicked (Bubbling Phase)");
});

outerBox.addEventListener(
    "click",
    function () {
        console.log("Outer DIV clicked (Capturing Phase)");
    },
    true
);

input.addEventListener("click", function (e) {
    e.stopPropagation();
    console.log("Input clicked -- Bubbling stopped!");
});
</script>
</body>
</html>

```